
Lab Assignment 10.2

Program	: B. Tech (CSE)
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	: III
Academic Session	: 2025-2026
Name of Student	: Sankeerthana
Enrollment No.	: 2403A51L41
Batch No.	: 52
Date	: 10-02-2026

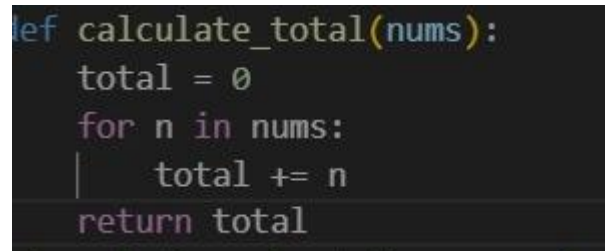
Task 1:

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Prompt: Find the errors in given code

Corrected Code:



```
def calculate_total(nums):
    total = 0
    for n in nums:
        total += n
    return total
```

Explanation:

The error was due to inconsistent variable naming, which led to a NameError.

The variable 'sum' was used instead of 'total', which caused the function to fail when trying to return 'total'.

Task 2:

Sample Input Code:

```
def findSum(a,b):return a+b
print(findSum(5,10))
```

Prompt: Give the well structured code with standard coding styles

Code:

```
def find_sum(a, b):  
    return a + b  
print(find_sum(5, 10))
```

Explanation:

The refactored code follows standard coding style guidelines by using snake_case for the function name and parameters, which is a common convention in Python. The function name 'find_sum' is more descriptive and easier to read than 'findSum'. Additionally, the code is formatted with proper spacing around operators and after commas, enhancing readability.

Task 3:

Sample Input Code:

```
def f(x,y):  
return x-y*2  
print(f(10,3))
```

Prompt: Improve code readability without changing its functionality.

Code:

```
def f(x, y):  
    return x - y * 2  
print(f(10, 3))
```

Explanation:

The improved code adds proper indentation to the function definition, which is essential for Python syntax.

It also includes spaces around the operators for better readability, making it easier to understand the mathematical operation being performed.

Task 4:

Sample Input Code:

```
print("Hello Ram")  
print("Hello Sita")  
print("Hello Ravi")
```

Prompt: Make refactor repetitive code into reusable functions that should eliminate repetition

Code:

```
def greet(name):  
    print(f"Hello {name}")  
greet("Ram")  
greet("Sita")  
greet("Ravi")
```

Explanation:

The refactored code defines a reusable function 'greet' that takes a name as an argument and prints a greeting message. This eliminates the repetition of the print statements for each name, making the code more concise and easier to maintain. By using a function, we can easily greet any number of names without having to write multiple print statements.

Task 5:

Sample Input Code:

```
numbers = [ ]  
for i in range(1, 500000):  
    numbers.append(i * i)  
print(len(numbers))
```

Prompt: Optimize code for better performance that achieves the same result with improved performance

Code:

```
numbers = [i * i for i in range(1, 500000)]  
print(len(numbers))
```

Explanation:

The original code uses a for loop to append the squares of numbers from 1 to 499999 to a list, which can be inefficient due to the overhead of multiple append operations. The optimized code uses a list comprehension, which is generally faster and more concise for creating lists.